

INSTALLATION GUIDE

# SNOMED CT Terminology Server

A user-friendly setup guide for running the SNOMED CT Terminology Server for IRIS for Health with MCP tools.

**Recommended path:** Use the full image for the fastest setup. Use the base image when you need to load your own RF2 data and generate embeddings.

# Table of Contents

Overview

Prerequisites

Option A: Full Image (Recommended)

1. Load the Docker Images
2. Create a Docker Compose Override
3. Pull the Embedding Model
4. Verify the Installation

Option B: Base Image (Build Your Own Data)

1. Load the Docker Images
2. Start the Containers
3. Pull the Embedding Model
4. Obtain SNOMED CT RF2 Files
5. Load the RF2 Data
6. Compute the Transitive Closure
7. Generate Vector Embeddings
8. Verify the Data

Configuring an LLM Client to Use the MCP Server

Available MCP Tools

Connecting via `iris-mcp-server` (stdio transport)

Connecting via SSE/HTTP transport

MCP Configuration File

Verifying MCP Connectivity

Troubleshooting

Ollama connection errors during semantic search

MCP tools not appearing in Claude Desktop/Code

Re-loading data

# Overview

---

This guide covers installing and running the SNOMED CT Terminology Server for IRIS for Health with MCP tools.

## Prerequisites

---

- [Docker Desktop](#) installed and running
  - At least 20GB free disk space (for the full image) or 8GB (for the base image)
  - A SNOMED CT RF2 release (required only for the base image path)
- 

## Option A: Full Image (Recommended)

---

The full image includes the US Edition with all data pre-loaded: concepts, descriptions, relationships, transitive closure, vector embeddings, and HNSW index. This is the fastest way to get started.

### 1. Load the Docker Images

```
docker load -i docker-images/snomed-iris-us-edition-2026.03.tar
docker load -i docker-images/ollama-latest.tar
```

**Note:** The Ollama image may not load from the `.tar` file due to platform differences. If so, pull it directly:

```
docker pull ollama/ollama:latest
```

### 2. Create a Docker Compose Override

Create a `docker-compose.override.yml` to use the pre-built full image instead of building from source:

```
services:
  iris:
    image: snomed-iris:us-edition-2026.03
    build: !override # disable build
```

Alternatively, run the containers directly:

```
# Start Ollama (needed for semantic search)
docker run -d --name ollama -p 11434:11434 -v ollama-data:/root/.ollama ollama/ollama:latest

# Start the SNOMED IRIS server
docker run -d --name snomed-iris \
  -p 1972:1972 \
  -p 52773:52773 \
  -e OLLAMA_URL=http://host.docker.internal:11434 \
  -v iris-data:/usr/irissys/mgr \
  snomed-iris:us-edition-2026.03
```

### 3. Pull the Embedding Model

The semantic search feature requires an embedding model loaded in Ollama:

```
docker exec ollama ollama pull all-minilm
```

### 4. Verify the Installation

Open the IRIS Management Portal at <http://localhost:52773/csp/sys/UtilHome.csp> (login: `_SYSTEM / SYS`).

Test MCP connectivity by navigating to **System Administration > Security > Applications > Web Applications** and confirming `/mcp/snomed` is registered.

---

## Option B: Base Image (Build Your Own Data)

The base image includes the compiled ObjectScript classes, MCP server configuration, and Python dependencies — but no SNOMED data. You will load RF2 data and generate embeddings yourself. This process takes approximately 2 hours.

### 1. Load the Docker Images

```
docker load -i docker-images/sct-iris-latest.tar
docker pull ollama/ollama:latest
```

## 2. Start the Containers

```
docker compose up -d
```

Or run individually:

```
docker run -d --name ollama -p 11434:11434 -v ollama-data:/root/.ollama ollama/ollama:latest
docker run -d --name snomed-iris \
  -p 1972:1972 \
  -p 52773:52773 \
  -e OLLAMA_URL=http://host.docker.internal:11434 \
  -v iris-data:/usr/irissys/mgr \
  sct-iris:latest
```

## 3. Pull the Embedding Model

```
docker exec ollama ollama pull all-minilm
```

## 4. Obtain SNOMED CT RF2 Files

You need a SNOMED CT RF2 Snapshot release. For the UK edition, download from [NHS TRUD](#). For the US edition, download from the [NLM UMLS](#).

Copy the RF2 files into the container:

```
docker cp /path/to/SnomedCT_Release/Snapshot/Terminology/ snomed-iris:/opt/snomed-data/
```

## 5. Load the RF2 Data

Open an IRIS terminal session:

```
docker exec -it snomed-iris iris session IRIS -U USER
```

Load concepts, descriptions, and relationships:

```
// Load concepts
Set sc = ##class(SNOMED.Loader).LoadConcepts("/opt/snomed-data/sct2_Concept_Snapshot_*.txt")
If $$$ISERR(sc) Do $SYSTEM.Status.DisplayError(sc)

// Load descriptions
Set sc = ##class(SNOMED.Loader).LoadDescriptions("/opt/snomed-data/sct2_Description_Snapshot_*.txt")
If $$$ISERR(sc) Do $SYSTEM.Status.DisplayError(sc)

// Load relationships
Set sc = ##class(SNOMED.Loader).LoadRelationships("/opt/snomed-data/sct2_Relationship_Snapshot_*.txt")
If $$$ISERR(sc) Do $SYSTEM.Status.DisplayError(sc)
```

**Note:** Replace \* with the actual date stamp in the filenames (e.g., sct2\_Concept\_Snapshot\_INT\_20240101.txt ). Progress is reported every 50,000 records.

## 6. Compute the Transitive Closure

This builds the hierarchy indexes for O(1) subsumption checking and ancestor/descendant queries:

```
Set sc = ##class(SNOMED.TransitiveClosure).Compute()
If $$$ISERR(sc) Do $SYSTEM.Status.DisplayError(sc)
```

This processes all active IS-A relationships and populates the ^SNOMED.TC and ^SNOMED.Children globals.

## 7. Generate Vector Embeddings

This generates 384-dimensional embeddings for all active Fully Specified Names using Ollama. This is the longest step (typically 1-2 hours for 831K+ concepts):

```
Set sc = ##class(SNOMED.Embeddings).Generate("all-minilm", 100, "http://ollama:11434")
If $$$ISERR(sc) Do $SYSTEM.Status.DisplayError(sc)
```

Parameters: - "all-minilm" — the Ollama embedding model (384 dimensions) - 100 — batch size (number of terms per API call) - "http://ollama:11434" — Ollama URL (use container hostname when running in Docker network)

**Resume support:** If the process is interrupted, re-run the same command. It automatically skips concepts that already have embeddings.

## 8. Verify the Data

```
// Check record counts
Write ##class(%SQL.Statement).%ExecDirect(,"SELECT COUNT(*) FROM SNOMED.Concept").%Next(), !
Write ##class(%SQL.Statement).%ExecDirect(,"SELECT COUNT(*) FROM SNOMED.Description").
%Next(), !
Write ##class(%SQL.Statement).%ExecDirect(,"SELECT COUNT(*) FROM SNOMED.Relationship").
%Next(), !
Write ##class(%SQL.Statement).%ExecDirect(,"SELECT COUNT(*) FROM SNOMED.ConceptEmbedding").
%Next(), !
```

# Configuring an LLM Client to Use the MCP Server

The SNOMED Terminology Server exposes its tools via the Model Context Protocol (MCP). Any MCP-compatible client can connect.

## Available MCP Tools

| Tool                                  | Description                                      |
|---------------------------------------|--------------------------------------------------|
| GetConcept(sctid)                     | Look up a concept by SNOMED CT identifier        |
| LexicalSearch(term, maxResults)       | Full-text keyword search across descriptions     |
| SearchByPattern(pattern, maxResults)  | SQL LIKE pattern search (use % wildcards)        |
| SemanticSearch(query, maxResults)     | Find concepts by meaning using vector similarity |
| GetParents(conceptId)                 | Get direct parent concepts                       |
| GetChildren(conceptId)                | Get direct child concepts                        |
| GetAncestors(conceptId)               | Get all ancestor concepts                        |
| GetDescendants(conceptId, maxResults) | Get all descendant concepts                      |
| IsA(conceptId, ancestorId)            | Check if a concept is a descendant of another    |

## Connecting via `iris-mcp-server` (stdio transport)

The `iris-mcp-server` binary bridges LLM clients to the IRIS MCP endpoint. Download it from the [InterSystems releases](#).

### Claude Desktop

Add to `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS) or `%APPDATA%\Claude\claude_desktop_config.json` (Windows):

```
{
  "mcpServers": {
    "snomed": {
      "command": "/path/to/iris-mcp-server",
      "args": ["--config=/path/to/config/mcp-config.toml", "run"]
    }
  }
}
```

### Claude Code

Add to your project's `.mcp.json` or `~/.claude/settings.json`:

```
{
  "mcpServers": {
    "snomed": {
      "command": "/path/to/iris-mcp-server",
      "args": ["--config=/path/to/config/mcp-config.toml", "run"]
    }
  }
}
```

## Connecting via SSE/HTTP transport

If your LLM client supports HTTP-based MCP (e.g., remote deployment), configure it to connect directly to the IRIS web server:

```
URL: http://localhost:52773/mcp/snomed
```

## MCP Configuration File

The `config/mcp-config.toml` template controls how `iris-mcp-server` connects to IRIS:



```
[mcp]
transport = "stdio"

[[iris]]
name = "local"

[iris.server]
host = "localhost"
port = 1972
username = "CSPSystem"
password = "SYS"

[iris.pool]
min = 2
max = 5

[[iris.endpoints]]
path = "/mcp/snomed"
```

Adjust `host`, `port`, `username`, and `password` to match your environment. If IRIS is running in Docker, `localhost` and port `1972` are correct when using the default port mapping.

## Verifying MCP Connectivity

Once configured, test from your LLM client by asking:

- “What is SNOMED concept 73211009?”
- “Search for concepts related to diabetes”
- “Find the children of concept 73211009”
- “Search semantically for broken arm bone”

---

## Troubleshooting

### Ollama connection errors during semantic search

Ensure Ollama is running and the `all-minilm` model is pulled:

```
docker exec ollama ollama list
```

If running IRIS outside Docker, set the `OLLAMA_URL` environment variable:

```
export OLLAMA_URL=http://localhost:11434
```

## MCP tools not appearing in Claude Desktop/Code

1. Verify `iris-mcp-server` is installed and the path in your config is correct
2. Check that IRIS is running: `docker ps | grep snomed-iris`
3. Verify the `/mcp/snomed` web application is registered in the Management Portal
4. Check logs: `iris-mcp-snomed.log` (location depends on your config)

## Re-loading data

To clear all data and start fresh:

```
Do ##class(SNOMED.Loader).TruncateAll()  
Do ##class(SNOMED.TransitiveClosure).Clear()
```

Then repeat steps 5-7 from Option B.